

Project 1: Build and Evaluate a Small Code Language Model

CS 5090 · Software Engineering with Generative AI

Individual Project · 25% of Course Grade · Due Friday, October 30

1. Overview

In this project, you will train a small code language model from scratch, use it on a software engineering task, study how it fails, and make one evidence-based improvement. The goal is not to build a strong production model. The model is intentionally small so that you can understand the full workflow and inspect its behavior.

The project has four phases:

Phase	Recommended pace	Main task	Deliverable
1. Build & understand	by Sep 25	Train a small code LM and run one ablation	Model card, data sheet, repository
2. Use	by Oct 9	Use your model on 20 real SE inputs	Task report
3. Evaluate & break	by Oct 16	Design probes and document 10+ distinct failures	Findings table (CSV)
4. Improve & re-evaluate	by Oct 23	Make one change and re-run the same probes	Improvement report, demo video

All components are submitted together on Friday, October 30. The intermediate dates are recommended milestones, not separate deadlines.

What to submit

- Model card and data sheet
- Code repository with reproducible training/configuration files
- Phase 2 task report
- Phase 3 findings table (CSV)
- Phase 4 improvement report
- 3-minute demo video
- AI-use log
- One-page handwritten AI-literacy reflection

2. Phase 1 — Build & Understand

Train a small GPT-style model from scratch. You may use either [nanoGPT](#) (recommended) or [llama2.c](#). Find your own Python corpus, for example a few permissively licensed repositories whose code you know well enough to judge; different corpora across the class make the results more diverse. Keep collection light (a few hours, not a project of its own), and document in your data sheet how the data was filtered, deduplicated, tokenized, and split.

Example model configuration

The following is one reasonable configuration for this project. You may adjust these settings based on your codebase, compute environment, or experimental goals. Document any major changes in your model card.

Setting	Value
Layers / heads / embedding size	6 / 6 / 384
Context length	512 tokens
Tokenizer	BPE, vocabulary 8,192; train only on the training split
Training tokens	approximately 20 × parameter count
Precision	bf16 when supported

Recommended ablation options

After the baseline model trains, choose one ablation. The options below are recommended; you may propose a comparable alternative with instructor approval. Write down your prediction before running it, then retrain and compare final validation loss and one side-by-side generation sample.

Recommended option	Question it helps you study
Remove position embeddings	Why does token order need to be represented?
Use 1 attention head instead of 6	What is gained by multiple heads?
Use 3 layers instead of 6	What is gained by depth?
Remove LayerNorm	How does normalization affect training stability?
Context length 256 instead of 512	What does a longer context help with?
Character-level tokenizer instead of BPE	How does tokenization change training and generation?

You should be able to explain

- How source code becomes tokens and why tokenizer choice matters.
- How embeddings and self-attention provide contextual token representations.
- What next-token prediction and cross-entropy loss optimize.
- The difference between pre-training and inference.
- How to interpret training and validation loss curves.
- Why a small model trained on a small corpus produces fluent but incorrect code.

Phase 1 deliverables

- Repository containing data preparation, training configuration, and instructions to reproduce the baseline and ablation runs.
- Model card (Appendix A).
- Data sheet (Appendix B).

If your training run fails after reasonable effort, contact the instructor about the spare checkpoint. You can still complete Phases 2–4, but you will not receive credit for missing Phase 1 work.

3. Phase 2 — Use the Model on a Real SE Task

Use your own model on one software engineering task. Examples include code completion, docstring generation, test skeleton generation, or another task approved by the instructor. Use 20 real inputs from code that you understand well enough to judge.

For each input, classify the result as:

- Accept as-is
- Accept after editing
- Reject

Keep three representative examples: your best result, your worst result, and one near-miss that initially looked correct but was not.

Phase 2 deliverable

Submit a 1-2 page task report using Appendix C. Report the three counts above and briefly explain what your model was useful for and when checking its output cost more time than writing the code yourself.

4. Phase 3 — Evaluate & Break

Design a probe set that deliberately tests your model. Include ordinary inputs and cases that may expose weaknesses, such as misleading comments or names, long-context inputs, security-related prompts, or prompts similar to the training data.

Find at least 10 distinct failures. Record each failure in the findings table (Appendix D). Use the following category names exactly so results can be compared across the class.

Category	Definition
syntax	Output does not parse
invented-api	Calls an API or function that does not exist
wrong-logic	Parses/runs but produces the wrong behavior
insecure	Introduces a security weakness
memorised	Reproduces training data nearly verbatim
prompt-sensitive	A small rewording changes the answer substantially
context-ignored	Ignores information clearly present in the input
other	Use only when none of the above applies; explain clearly

For every failure, include a specific one-line explanation of why it is wrong and whether a normal unit-test suite would likely catch it.

5. Phase 4 — Improve & Re-evaluate

Choose the failure category that appears most often in your Phase 3 findings. Form a hypothesis about its cause, make one change, and re-run the same probes.

Possible intervention points

Area	Examples
Data	Remove unparseable files, improve deduplication, rebalance data
Tokenizer	Change vocabulary size or retrain on cleaner data
Training	More tokens, longer context, different learning-rate schedule
Fine-tuning	A small supervised fine-tuning pass on relevant examples
Decoding	Adjust temperature/top-p or sample multiple candidates

Experimental rules

1. State your hypothesis before running the experiment.
2. Change one thing only.
3. Re-run the same Phase 3 probes with the same decoding settings.
4. Report both improvements and regressions.
5. Report compute cost and validation loss when applicable.
6. A negative result is acceptable if it is measured and reported clearly.

Phase 4 deliverables

- A 2–3 page improvement report using Appendix E.
- Updated code/configuration in your repository.
- A 3-minute recorded demo showing one probe before and after the change, plus your main quantitative result.

6. AI-Use Log

You may use an AI coding assistant while completing the project. Keep a short log of meaningful interactions: what you asked, what you accepted, what required correction, and how you verified the result. The goal is to document where AI was useful and where it created extra work or confusion. Use Appendix F.

7. Handwritten Reflection

At the end of the project, complete the one-page reflection in Appendix G by hand, without an AI tool. Scan or photograph the page and include it in your submission. It is graded for completion, not for giving a particular answer.

8. Grading

Component	Weight	Full credit requires
Build & understand	25%	Baseline + ablation are reproducible; prediction recorded before ablation; model card and data sheet show understanding
Use	15%	20 real inputs; honest accept/edit/reject counts; three representative examples
Evaluate & break	25%	10+ distinct failures; correct categories; specific explanations; test-suite column completed
Improve & re-evaluate	25%	Clear hypothesis; one change; same probes re-run; regressions and cost reported
AI-use log, reflection, engineering quality	10%	Complete log and handwritten reflection; repository is reproducible and results are traceable

Model quality itself is not graded. A small model that produces poor code is still useful for this project if you analyze it carefully. Likewise, your Phase 4 change does not need to improve the model; a well-designed negative result can receive full credit.

9. Compute

A GPU is strongly recommended for Phase 1 training. Expect roughly 2–6 GPU-hours per successful run and plan for 2–3 runs across the project. Kaggle, Google Colab, or a local GPU are all acceptable. Save checkpoints regularly and log training loss so you can reproduce and analyze your runs later.

If an ACCESS classroom allocation becomes available, instructions will be posted separately. The project does not depend on ACCESS availability.

Appendices — Submission Templates

You may copy these sections into Markdown files in your repository and fill them in.

Appendix A — Model Card

- Model details: architecture, parameters, context length, vocabulary, seed, codebase commit, repository commit.
- Intended use: course project / teaching artifact; not for unreviewed production code.
- Training data: source, licenses, token count, deduplication, split strategy (refer to the data sheet for details).
- Evaluation: validation loss, syntactic-validity rate, any other benchmark, and decoding settings.
- Ablation: exact change, prediction written before the run, baseline vs. ablation loss, one sample comparison, and what you learned.
- Limitations: measured failure modes, Python-only scope, memorization/contamination checks if performed.
- Ethics/licensing/caveats: data licenses, secret/PII checks, and anything important that the headline metrics do not show.

Appendix B — Data Sheet

- Motivation and source of the corpus.
- Composition: file/token counts and licensing filters.
- Collection: snapshot/commit and exclusions.
- Preprocessing: cleaning steps, deduplication method and counts, tokenizer and vocabulary.
- Splits: train/validation/test sizes; repository-level split; how leakage was checked.
- Uses and limitations, including contamination risk.
- Distribution: whether the data can be redistributed; if not, provide a reproducible recipe.

Appendix C — Phase 2 Task Report

Length: 1-2 pages. Include your name, task, source of inputs, and why you can judge the outputs.

Outcome	Count
Accept as-is	
Accept after editing	
Reject	

- Define what “accept” means for your task.
- Show three examples: best, worst, and near-miss.
- What was the model actually useful for?
- When did checking the model take more time than writing the code yourself?

Appendix D — Findings Table (CSV)

Minimum 10 distinct failures. Use this exact header:

```
finding_id,category,what_you_gave_it,what_it_produced,why_wrong,severity,how_found,caught_by_test_suite
```

Allowed categories: syntax, invented-api, wrong-logic, insecure, memorised, prompt-sensitive, context-ignored, other. Severity: low / medium / high. For caught_by_test_suite, use yes / no.

Appendix E — Improvement Report

7. Target failure category and why you selected it.
8. Hypothesis (written before the run).
9. One change, described reproducibly.
10. Before/after comparison on the same probe set.
11. Compute cost, validation loss, and regressions.
12. Did the change work? Did it work for the reason you expected?
13. What would you try next?

Appendix F — AI-Use Log

Date	Tool	What you asked	What you accepted	What you fixed	How you verified

At the end, briefly reflect on: where AI helped, one confidently wrong result and how you caught it, whether AI reduced your understanding anywhere, and what you will do differently in Project 2.

Appendix G — Handwritten AI-Literacy Reflection

Print this page and complete it by hand without AI assistance.

Topic	Before (1-5)	Now (1-5)
How code becomes tokens and why tokenization matters		
What embeddings and self-attention do		
What next-token prediction / cross-entropy optimize		
Difference between training and inference		
How to read training vs. validation loss		
Why fluent output can still be wrong		

Answer each question in 1-2 handwritten sentences:

14. What can you explain now that you could not explain before this project?
 15. Did building and testing your own model change how you judge tools such as Copilot or ChatGPT? How?
 16. Which phase taught you the most, and why?
 17. What about language models still confuses you?
- Optional: one change that would have helped you learn more from this project.